

Министерство образования и науки РФ
Санкт-Петербургский Политехнический Университет Петра Великого
Институт Компьютерных Наук и Кибербезопасности
Высшая школа программной инженерии

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

по дисциплине

«Языки моделирования и описания цифровой аппаратуры»

Программная и аппаратная реализация алгоритма сортировки вставками

Выполнили:
студенты гр 5130904/40003

Лузин К.Л.
Николаев А.Д.
Парфенов Я.М.

Проверил:
ст. преподаватель

Петров А.В.

Оглавление

1. Техническое задание.....	3
2. Описание алгоритма.....	3
3. Аппаратная реализация.....	4
3.1. VHDL код.....	4
3.2. Симуляция (тестирование).....	5
3.3. Технологическая схема.....	6
3.4. RTL схема.....	7
3.5. Отчёт в среде Quartus II.....	8
4. Программная реализация алгоритма на VHDL-моделях DP32 и памяти.....	9
4.1. Блок схема.....	9
4.2. Исполняемая программа в машинном коде.....	10
4.3. Исполняемая программа на языке ассемблера DP32.....	11
4.4. Скриншоты тестирования ко-симуляцией.....	12
4.5. Ручной расчет.....	15
Алгоритм.....	15
Результат.....	16

1. Техническое задание

- 1) Разработать аппаратную реализацию сортировки пузырьком массива на языке VHDL
- 2) Произвести симуляцию VHDL кода
- 3) Создать RTL схему в среде Quartus II
- 4) Создать технологическую схему
- 5) Разработать программную реализацию сортировки вставками массива в кодах процессора DP32
- 6) Создать блок-схему алгоритма на кодах процессора DP32
- 7) Произвести ко-симуляцию в среде Active VHDL
- 8) Сделать ручной расчёт сортировки

2. Описание алгоритма

На вход алгоритма подаётся последовательность n чисел. Алгоритм состоит из прохода по массиву. На каждом шаге следующий элемент сравнивается с текущим, если порядок неверный, то следующий элемент вставляется в отсортированную часть. Проход по массиву проводится однократно, на каждом шаге отсортированная часть увеличивается, а не отсортированная уменьшается.

3. Аппаратная реализация

3.1. VHDL код

```
0 | library ieee;
1 | use ieee.std_logic_1164.all;
2 | use ieee.std_logic_arith.all;
3 | use ieee.std_logic_unsigned.all;
4 | package types is
5 |     type integer_array is array (natural range <>) of integer;
6 | end package types;
7 | package body types is
8 | end package body types;
9 |
10| library ieee;
11| use ieee.std_logic_1164.all;
12| use ieee.std_logic_arith.all;
13| use ieee.std_logic_unsigned.all;
14| use work.types.all;
15|
16| entity insertSort is
17|     generic
18|     (
19|         elementsNum : integer := 8
20|     );
21|     port
22|     (
23|         clk : in std_logic;
24|         start : in std_logic;
25|         reset : in std_logic;
26|         inArr : in integer_array(0 to (elementsNum - 1));
27|
28|         outArr : out integer_array(0 to (elementsNum - 1));
29|         done : out std_logic;
30|         working : out std_logic
31|     );
32| end insertSort;
33|
34| architecture behav of insertSort is
35|     signal blocking : std_logic;
36|     begin
37|         process(clk)
38|             variable localArr : integer_array(0 to (elementsNum - 1)) := (others => 0);
39|             variable temp : integer := 0;
40|             variable i : integer range 0 to (elementsNum - 1) := 0;
41|             variable j : integer range -1 to (elementsNum - 1) := 0;
42|         begin
43|             if (rising_edge(clk)) then
44|                 if (reset = '1') then
45|                     outArr <= (others => 0);
46|                     done <= '0';
47|                     blocking <= '1';
48|                 elsif (start = '1') then
```

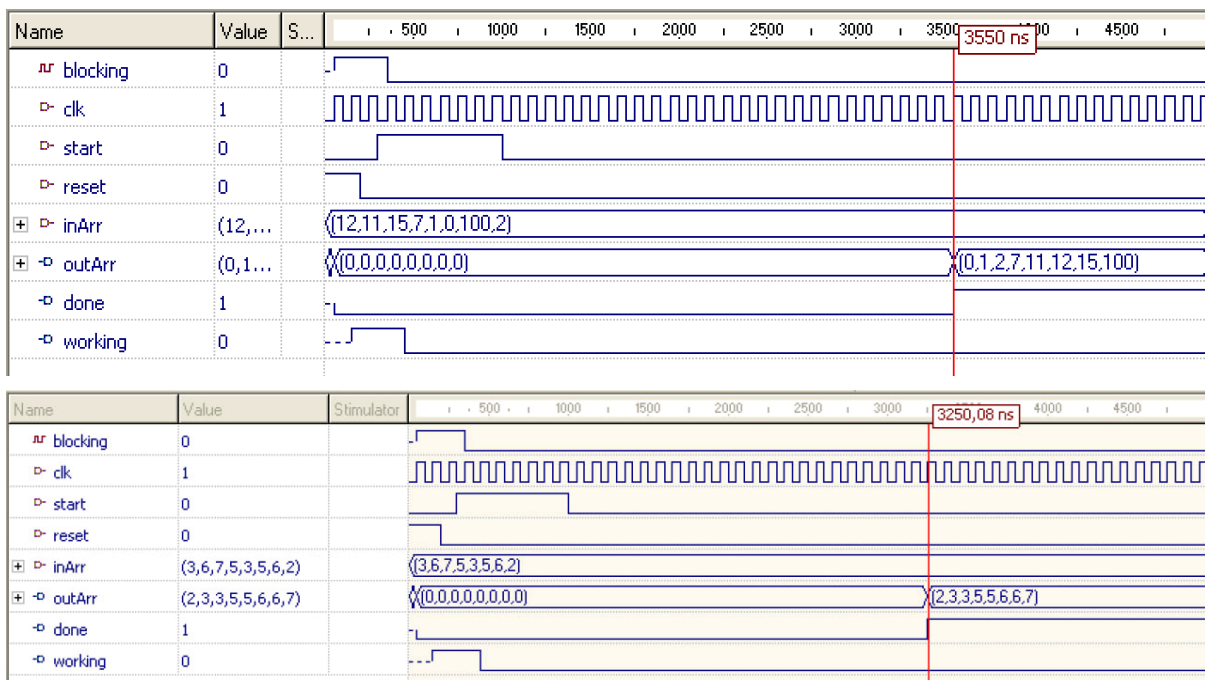
```

49|         outArr <= (others => 0);
50|         done <= '0';
51|         blocking <= '0';
52|         localArr := inArr;
53|         i := 0;
54|         j := 0;
55|     elsif (blocking = '0') then
56|         if (i < (elementsNum - 1)) then
57|             if ((j >= 0) and (localArr(j) > localArr(j + 1))) then
58|                 temp := localArr(j);
59|                 localArr(j) := localArr(j + 1);
60|                 localArr(j + 1) := temp;
61|                 j := j - 1;
62|             else
63|                 j := i + 1;
64|                 i := i + 1;
65|             end if;
66|         else
67|             done <= '1';
68|             outArr <= localArr;
69|             blocking <= '0';
70|         end if;
71|     end if;
72|     working <= blocking;
73| end if;
74| end process;
75| end behav;
76|
77|

```

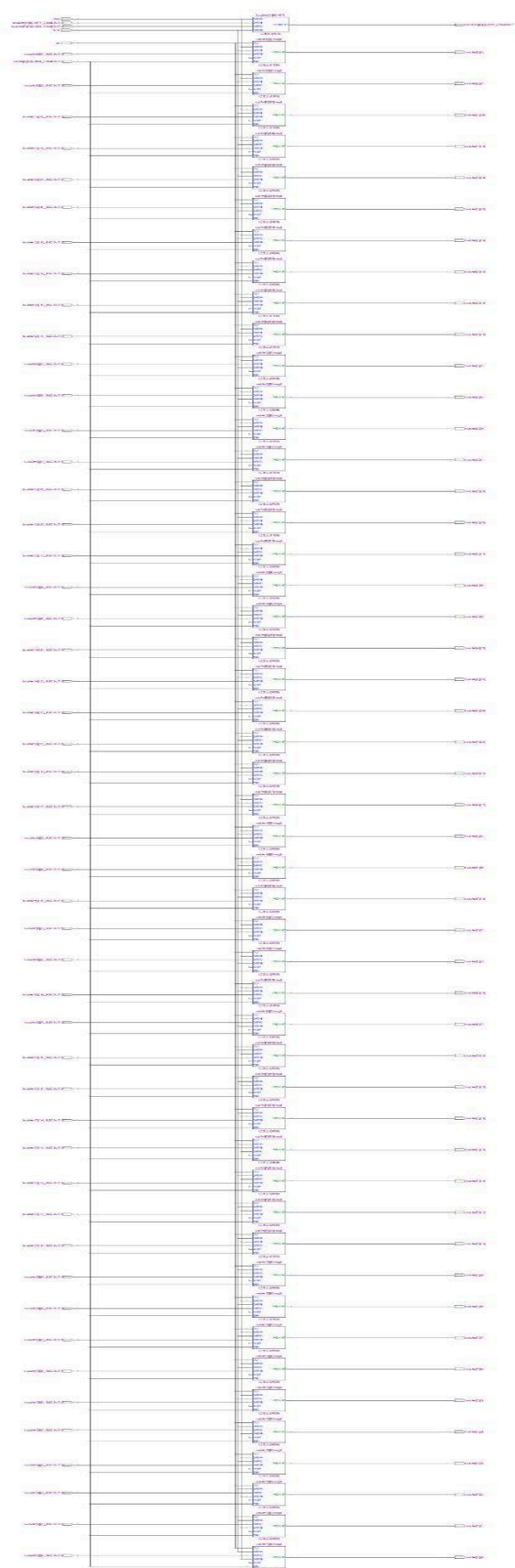
3.2. Симуляция (тестирование)

Waveform – диаграммы



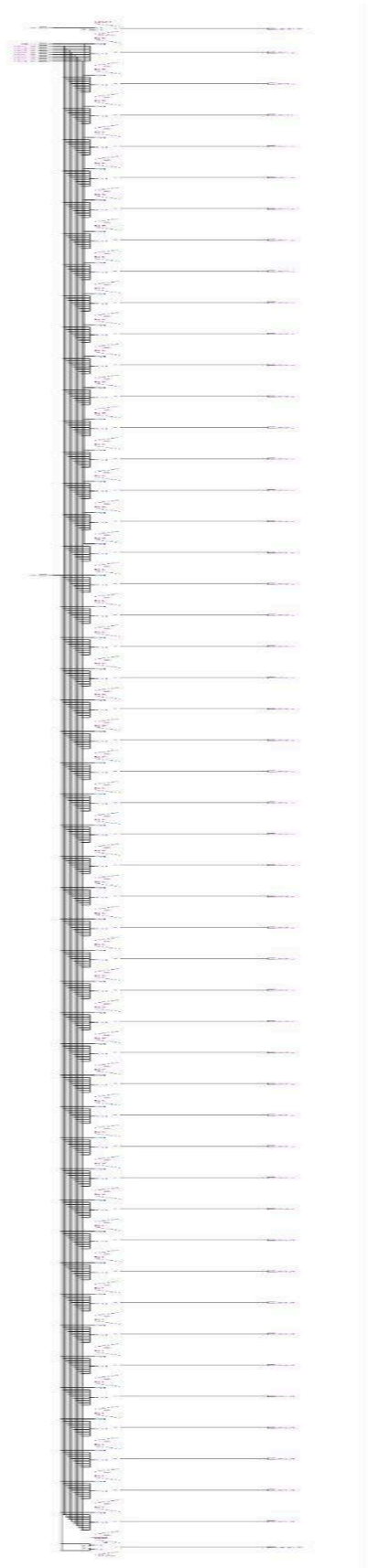
3.3. Технологическая схема

Технологическая схема создана на устройстве Stratex



3.4. RTL схема

RTL схема создана на устройстве Stratex



3.5. Отчёт в среде Quartus II

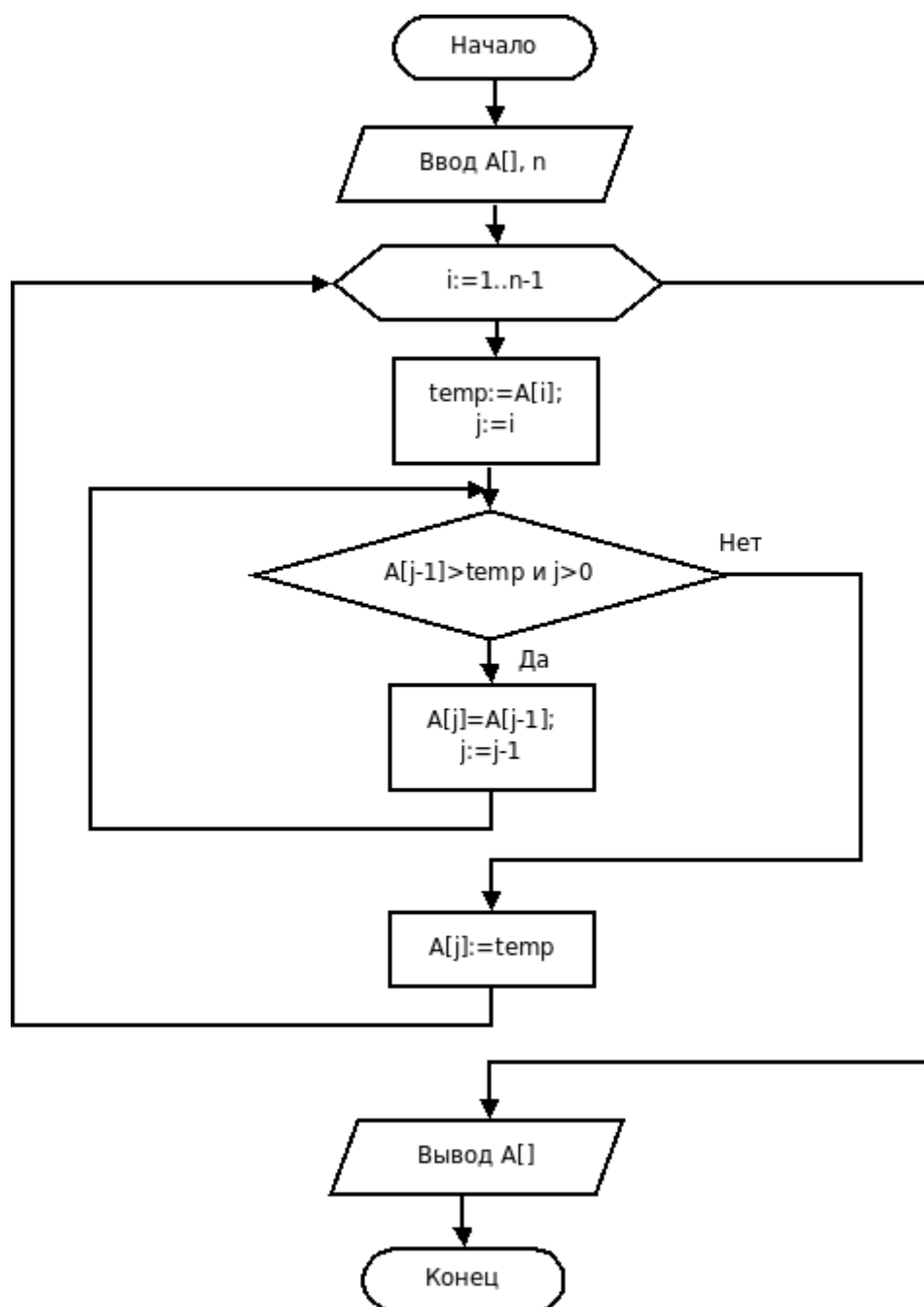
Quartus II Version	5.0 Build 148 04/26/2005 SJ Full Version
Revision Name	insertSort
Top-level Entity Name	insertSort
Family	Stratix
Met timing requirements	Yes
Total logic elements	909 / 18,460 (4 %)
Total pins	517 / 587 (88 %)
Total virtual pins	0
Total memory bits	0 / 1,669,248 (0 %)
DSP block 9-bit elements	0 / 80 (0 %)
Total PLLs	0 / 6 (0 %)
Total DLLs	0 / 2 (0 %)
Device	EP1S20F780C5
Timing Models	Final

Module	Progress %	Time 
Full Compilation	100 %	00:00:22
Analysis & Synthesis	100 %	00:00:06
Fitter	100 %	00:00:12
Assembler	100 %	00:00:02
Timing Analyzer	100 %	00:00:02

Analysis & Synthesis Resource Usage Summary			
	Resource ▼	Usage	
1	Total registers	522	
2	Total logic elements	909	
3	Total logic cells in carry chains	32	
4	Total fan-out	4810	
5	Total combinational functions	908	
6	Maximum fan-out node	start	
7	Maximum fan-out	535	
8	I/O pins	517	
9	Combinational cells for routing	0	
10	Average fan-out	3.37	
11	-- Total 4-input functions	277	
12	-- Total 3-input functions	591	
13	-- Total 2-input functions	39	
14	-- Total 1-input functions	0	
15	-- Total 0-input functions	1	

4. Программная реализация алгоритма на VHDL-моделях DP32 и памяти

4.1. Блок схема



4.2. Исполняемая программа в машинном коде

0700_0000
1009_000C
3109_0023
1009_000B
3109_0024
1009_000F
3109_0025
1009_0007
3109_0026
1009_0001
3109_0027
1009_0000
3109_0028
1009_0064
3109_0029
1009_0002
3109_002A

1003_0000
1003_0301
1109_0308
5009_00EB
3004_0323
1007_0300
3005_0723
1108_0701
1009_0700
500B_0006
3006_0823
0109_0604
500B_0003
3106_0723
1007_0800
5000_00F6
3104_0723
5000_00EF
0000_000C
0000_000B
0000_000F
0000_0007
0000_0001
0000_0000
0000_0064
0000_0002

4.3. Исполняемая программа на языке ассемблера DP32

```
r0 < r0 & !r0
r9 < r0 + C
M[r0 + 23] < r9
r9 < r0 + B
M[r0 + 24] < r9
r9 < r0 + F
M[r0 + 25] < r9
r9 < r0 + 7
M[r0 + 26] < r9
r9 < r0 + 1
M[r0 + 27] < r9
r9 < r0 + 0
M[r0 + 28] < r9
r9 < r0 + 64
M[r0 + 29] < r9
r9 < r0 + 2
M[r0 + 2A] < r9

r3 < r0 + 0
r3 < r3 + 1
r9 < r3 - 8
if (1=V&0|N&0|Z&1) then PC < PC + (-21)
r4 < M[r3 + 23]
r7 < r3 + 0
r5 < M[r7 + 23]
r8 < r7 - 1
r9 < r7 + 0
if (1=V&0|N&1|Z&1) then PC < PC + 6
r6 < M[r8 + 23]
r9 < r6 - r4
if (1=V&0|N&1|Z&1) then PC < PC + 3
M[r7 + 23] < r6
r7 < r8 + 0
if (0=V&0|N&0|Z&0) then PC < PC + (-10)
M[r7 + 23] < r4
if (0=V&0|N&0|Z&0) then PC < PC + (-17)
```

4.4. Скриншоты тестирования ко-симуляцией

Name	Type	Value	Last Value
+ mem/line__15/mem(42)	bit_32	00000002	-----
+ mem/line__15/mem(41)	bit_32	00000064	-----
+ mem/line__15/mem(40)	bit_32	00000000	-----
+ mem/line__15/mem(39)	bit_32	00000001	-----
+ mem/line__15/mem(38)	bit_32	00000007	-----
+ mem/line__15/mem(37)	bit_32	0000000F	-----
+ mem/line__15/mem(36)	bit_32	0000000B	-----
+ mem/line__15/mem(35)	bit_32	0000000C	-----

Name	Type	Value	Last Value
+ mem/line__15/mem(42)	bit_32	00000002	-----
+ mem/line__15/mem(41)	bit_32	00000064	-----
+ mem/line__15/mem(40)	bit_32	00000000	-----
+ mem/line__15/mem(39)	bit_32	00000001	-----
+ mem/line__15/mem(38)	bit_32	00000007	-----
+ mem/line__15/mem(37)	bit_32	0000000F	-----
+ mem/line__15/mem(36)	bit_32	0000000C	-----
+ mem/line__15/mem(35)	bit_32	0000000B	-----

	Name	Type	Value	Last Value
+	mem/line__15/mem(42)	bit_32	000000002	-----
+	mem/line__15/mem(41)	bit_32	000000064	-----
+	mem/line__15/mem(40)	bit_32	000000000	-----
+	mem/line__15/mem(39)	bit_32	000000001	-----
+	mem/line__15/mem(38)	bit_32	00000000F	-----
+	mem/line__15/mem(37)	bit_32	00000000C	-----
+	mem/line__15/mem(36)	bit_32	00000000B	-----
+	mem/line__15/mem(35)	bit_32	000000007	-----

	Name	Type	Value	Last Value
+	mem/line__15/mem(42)	bit_32	000000002	-----
+	mem/line__15/mem(41)	bit_32	000000064	-----
+	mem/line__15/mem(40)	bit_32	000000000	-----
+	mem/line__15/mem(39)	bit_32	00000000F	-----
+	mem/line__15/mem(38)	bit_32	00000000C	-----
+	mem/line__15/mem(37)	bit_32	00000000B	-----
+	mem/line__15/mem(36)	bit_32	000000007	-----
+	mem/line__15/mem(35)	bit_32	000000001	-----

	Name	Type	Value	Last Value
+	mem/line__15/mem(42)	bit_32	00000002	-----
+	mem/line__15/mem(41)	bit_32	00000064	-----
+	mem/line__15/mem(40)	bit_32	0000000F	-----
+	mem/line__15/mem(39)	bit_32	0000000C	-----
+	mem/line__15/mem(38)	bit_32	0000000B	-----
+	mem/line__15/mem(37)	bit_32	00000007	-----
+	mem/line__15/mem(36)	bit_32	00000001	-----
+	mem/line__15/mem(35)	bit_32	00000000	-----

	Name	Type	Value	Last Value
+	mem/line__15/mem(42)	bit_32	00000064	-----
+	mem/line__15/mem(41)	bit_32	0000000F	-----
+	mem/line__15/mem(40)	bit_32	0000000C	-----
+	mem/line__15/mem(39)	bit_32	0000000B	-----
+	mem/line__15/mem(38)	bit_32	00000007	-----
+	mem/line__15/mem(37)	bit_32	00000002	-----
+	mem/line__15/mem(36)	bit_32	00000001	-----
+	mem/line__15/mem(35)	bit_32	00000000	-----

4.5. Ручной расчет

Алгоритм

Был написан простой алгоритм на языке Go:

```
1.package main
2.
3.import "fmt"
4.
5.func main() {
6.    arr := []int{12, 11, 15, 7, 1, 0, 100, 2}
7.
8.    fmt.Printf("%v\n\n", arr)
9.
10.   for i := 1; i < len(arr); i++ {
11.       j := i
12.       for j > 0 && arr[j-1] > arr[j] {
13.           arr[j-1], arr[j] = arr[j], arr[j-1]
14.           j--
15.
16.           fmt.Print("(")
17.           for k := 0; k < len(arr); k++ {
18.               if k == j || k == j+1 {
19.                   fmt.Printf("\033[7m%d\033[0m", arr[k])
20.               } else {
21.                   fmt.Printf("%d", arr[k])
22.               }
23.
24.               if k < len(arr)-1 {
25.                   fmt.Print(", ")
26.               }
27.           }
28.           fmt.Println(") ")
29.       }
30.   }
31.
32.   fmt.Printf("\n%v", arr)
33. }
```

Результат

Результаты работы алгоритма

[12 11 15 7 1 0 100 2]

(11, 12, 15, 7, 1, 0, 100, 2)

(11, 12, 7, 15, 1, 0, 100, 2)

(11, 7, 12, 15, 1, 0, 100, 2)

(7, 11, 12, 15, 1, 0, 100, 2)

(7, 11, 12, 1, 15, 0, 100, 2)

(7, 11, 1, 12, 15, 0, 100, 2)

(7, 1, 11, 12, 15, 0, 100, 2)

(1, 7, 11, 12, 15, 0, 100, 2)

(1, 7, 11, 12, 0, 15, 100, 2)

(1, 7, 11, 0, 12, 15, 100, 2)

(1, 7, 0, 11, 12, 15, 100, 2)

(1, 0, 7, 11, 12, 15, 100, 2)

(0, 1, 7, 11, 12, 15, 100, 2)

(0, 1, 7, 11, 12, 15, 2, 100)

(0, 1, 7, 11, 12, 2, 15, 100)

(0, 1, 7, 11, 2, 12, 15, 100)

(0, 1, 7, 2, 11, 12, 15, 100)

(0, 1, 2, 7, 11, 12, 15, 100)

[0 1 2 7 11 12 15 100]